

SENIOR LINUX TROUBLESHOOTING HANDBOOK

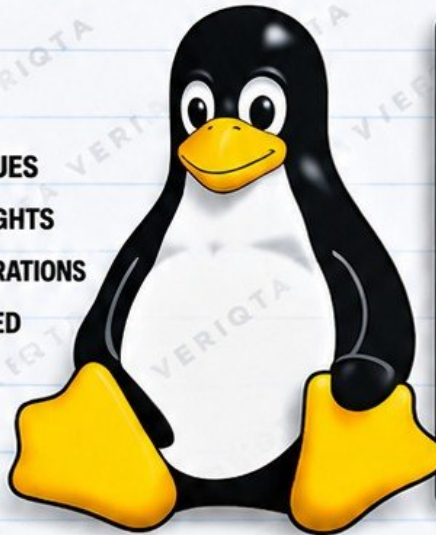
TROUBLESHOOT
ANALYZE
SOLVE
IMPROVE
REPEAT

DIAGNOSE | INVESTIGATE | RESOLVE | PREVENT

PRODUCTION-FOCUSED LINUX TROUBLESHOOTING
FOR SENIOR ENGINEERS

- ✓ REAL-WORLD SCENARIOS
- ✓ PRACTICAL INVESTIGATION STEPS
- ✓ ADVANCED DEBUGGING TECHNIQUES
- ✓ PERFORMANCE AND SCALING INSIGHTS
- ✓ SECURITY AND RELIABILITY CONSIDERATIONS
- ✓ EXPERT TIPS AND LESSONS LEARNED

*From Symptoms
to Root Cause*



```
[linux@server ~]$  
top  
journalctl  
dmesg  
ss -tulnp  
df -h  
free -m  
ps aux  
strace  
perf  
tcpdump  
[linux@server ~]$
```



PERFORMANCE
ISSUES



STORAGE
PROBLEMS



SERVICE
FAILURES



NETWORK
TROUBLESHOOTING



SECURITY
INVESTIGATION



DEEP
DIAGNOSTICS

A structured approach to investigate, understand and solve complex Linux production issues.

1 SYMPTOM vs ROOT CAUSE



- Symptoms are what you see.
- Root cause is the underlying reason.
- A single symptom can have multiple causes.
- Always validate, don't assume.

"Fixing the symptom without finding the cause leads to repeat incidents."

2 DEFINE BLAST RADIUS



- Identify what is affected (services, users, regions).
- Check if it's a single host, cluster or wider infrastructure.
- Understand dependencies.
- Prevent unnecessary changes outside the blast radius.

"Contain the problem before you fix it."

3 ESTABLISH INCIDENT TIMELINE



- Note when the issue started.
- Look at key events (deployments, config changes, scaling, failures).
- Correlate logs, metrics and alerts.
- Identify what changed just before the issue.

"Time tells a story. Find the turning point."

4 CHECK RECENT CHANGES



- Review deployments and releases.
- Check configuration changes.
- Look for infrastructure changes (servers, networking, storage).
- Validate recent package updates.
- Include changes from automation and other teams.

"Most production issues are caused by recent changes."

5 USE, RED AND GOLDEN SIGNALS



- USE: Utilization, Saturation, Errors (for infrastructure).
- RED: Rate, Errors, Duration (for services).
- Golden Signals: Latency, Traffic, Errors, Saturation.
- Use the right signals for the layer you're troubleshooting.

"Metrics show you what's happening, not what to do. Interpret them."

6 METRICS, LOGS, TRACES, KERNEL EVIDENCE



- Metrics: show trends and scale.
- Logs: show detailed events.
- Traces: show request flow across services.
- Kernel evidence: check dmesg, journalctl, /proc and system logs.
- Combine multiple sources for a complete picture.

"One data source is a clue. Multiple sources reveal the truth."

7 HYPOTHESIS-DRIVEN INVESTIGATION



- Form a hypothesis based on evidence.
- Test the highest-probability cause first.
- Gather data, don't guess.
- Narrow down the failure domain.
- Validate or reject the hypothesis and iterate.

"Investigate with a plan, not random commands."

8 MITIGATE, VERIFY, PREVENT



- Mitigate: apply the safest fix to restore service.
- Verify: confirm the issue is fully resolved and monitor for recurrence.
- Prevent: address the root cause, improve monitoring and update processes.

"Fix, verify, learn and make it stronger."

9 PRODUCTION PRINCIPLE: OBSERVE BEFORE CHANGING



- Gather evidence before making changes.
- Understand the impact of any action.
- Avoid unnecessary restarts or speculative fixes.
- In production, information is more valuable than action.

"Observe. Understand. Then change."

Linux System Triage Under Pressure

Get a clear picture fast. Be methodical. Make the right next move.

Real Systems
Real Problems
Real Solutions

1 FIRST 5 MINUTES OF AN INCIDENT



- ✓ Confirm the alert and impact.
- ✓ Understand what users are experiencing.
- ✓ Check recent changes (deployments, config, scaling, maintenance).
- ✓ Assess the blast radius (single host, service, cluster, region).
- ✓ Gather quick system signals (CPU, memory, disk, network).
- ✓ Build an initial hypothesis.

Speed is important, but accuracy is more important. Take a structured approach, not random commands.

2 KEY COMMANDS FOR INITIAL TRIAGE



<code>uptime</code>	# system load, uptime
<code>top</code>	# real-time process view
<code>vmstat 1 5</code>	# CPU, memory, io, context switch
<code>free -h</code>	# memory usage
<code>df -h</code>	# disk space usage
<code>dmesg tail</code>	# recent kernel messages

These commands give a quick view of system health. Use them together to understand the overall picture.

3 CPU, MEMORY, DISK, NETWORK QUICK ASSESSMENT



CPU

- Check CPU usage, load average, run queue.
- Identify top CPU-consuming processes.
- Look for high system % (kernel) or iowait.



MEMORY

- Check used, free and available memory.
- Look for swap usage.
- Identify memory-hungry processes.



DISK

- Check disk usage (`df -h`).
- Look for high I/O wait (`vmstat`, `iostat`).
- Identify processes doing heavy I/O (`iostat`).



NETWORK

- Check interface status (`ip link`, `ip addr`).
- Look for high traffic or errors (`ss`, `nload`).
- Confirm connectivity to key dependencies.

4 LOAD AVERAGE INTERPRETATION



Load average shows the number of runnable and uninterruptible tasks in the run queue (over 1, 5 and 15 minutes).

- Compare with the number of CPU cores.
- Load < cores → normal.
- Load ≈ cores → system is busy.
- Load > cores → CPU contention.
- High load with low CPU usage usually indicates iowait (disk) or uninterruptible processes.

Check `/proc/loadavg` for more details:
`cat /proc/loadavg`

5 DETECTING RESOURCE SATURATION



- CPU: high load average, high %us or %sy.
- Memory: high usage, swap activity, OOM risk.
- Disk: high I/O wait, high latency, 100% utilization.
- Network: high bandwidth, packet drops, errors.
- File descriptors: nearing limits (`ulimit`, `/proc`).
- Processes: unusual spikes or runaway processes.

Saturation in one resource can cause issues in others (e.g. disk I/O wait can make CPU look idle).

6 HOST FAILURE vs APPLICATION FAILURE



Likely host-level issue if:

- Multiple services are affected.
- High CPU, memory, disk or network usage.
- System commands are slow or failing.
- Kernel errors in `dmesg`.

Likely application-level issue if:

- Only one service is affected.
- System resources look normal.
- Other services on the host are healthy.
- Errors are in application logs.

7 BUILDING AN INITIAL FAILURE HYPOTHESIS



1. Gather the evidence (metrics, logs, system state).
2. Look for patterns and recent changes.
3. Form the most likely cause (hypothesis).
4. Test the hypothesis with targeted commands.
5. Refine as you get more data.

"A good engineer doesn't guess. They investigate, prove and eliminate possibilities."

1 CPU TIME STATES

Linux reports CPU time in different states. Understand what each one means.

State	Meaning
user	Time spent running user-space processes
system	Time spent in kernel (system calls, drivers)
idle	CPU is not doing useful work
iowait	Waiting for disk or network I/O
steal	CPU time taken by the hypervisor (VMs)

High % in user or system = CPU busy
High % in iowait or steal = CPU not available to your apps.

2 USEFUL COMMANDS

Use these commands to investigate CPU issues.

```
top           # real-time process view
htop          # interactive (if available)
pidstat -u 1   # per-process CPU usage
mpstat -P ALL 1 # per-CPU usage
vmstat 1       # overall system view
ps -eo pid,ppid,comm,%cpu --sort=-%cpu
sar -u 1 5     # CPU statistics (if sysstat)
```

Use multiple tools. One tool rarely tells the whole story.

3 RUN QUEUES

The run queue shows how many processes are waiting for CPU time.

- High run queue = more demand than CPU supply.
- Consistently high run queue indicates CPU saturation.
- Check per-CPU run queue with `mpstat` or `/proc/loadavg`.

```
$ cat /proc/loadavg
4.32 3.80 2.99 2/456 12345 12345
```

1 min average 5 min average 15 min average running/total processes last PID

4 CONTEXT SWITCHES

Context switches happen when the CPU switches between threads or processes.

- High context switches can indicate CPU contention.
- Measure with `vmstat` or `pidstat`.
- Very high values may indicate too many threads, lock contention or CPU limits.

```
$ vmstat 1 5
procs -----memory----- --swap-- --io-- --system--
r b swpd free buff cache si so bi bo in cs us sy id wa st
3 0 0 123456 23456 987654 0 0 12 34 5678 12345 20 10 65 3 2
```

High cs (context switches) with high CPU usage usually means CPU contention.

5 CPU AFFINITY

CPU affinity controls which CPUs a process can run on.

- Use `taskset` to set or check CPU affinity.
- Useful for performance tuning and isolation.

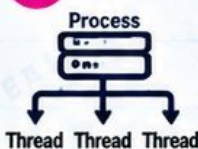
```
$ taskset -cp 1234 # check CPU affinity
$ taskset -c 0-3 1234 # run on specific CPUs
```

6 NICE VALUES AND PRIORITIES

- Nice values control process scheduling priority.
- Lower nice value = higher priority (more CPU).
- Higher nice value = lower priority (less CPU).
- Range: -20 (highest) to 19 (lowest).

```
$ nice -n 10 ./app # start with lower priority
$ renice -n -5 -p 1234 # change priority of a process
```

7 THREADS VERSUS PROCESSES



- A process can have multiple threads.
- Threads share the same memory space.
- High thread count can increase context switches and scheduling overhead.
- Use: `ps -T -p <pid>` or `top` (press H) to view threads.

8 HOT PROCESS vs OVERLOADED HOST

Hot Process

- One process uses high CPU.
- Other processes are normal.
- Fix: optimize, scale or limit that process.

Overloaded Host

- Many processes use CPU.
- High run queue.
- System feels slow overall.
- Fix: add CPU, reduce load, optimize workloads.

9 VIRTUALIZATION CPU STEAL



- steal** time is CPU time taken by the hypervisor.
- Common in virtualized/cloud environments.
- High steal time means your VM wants CPU but the host is not giving it.
- Check with `top`, `vmstat` or `mpstat`.
- Work with your cloud provider if steal time is high.

10 PRODUCTION SCENARIO



High load with apparently idle CPUs

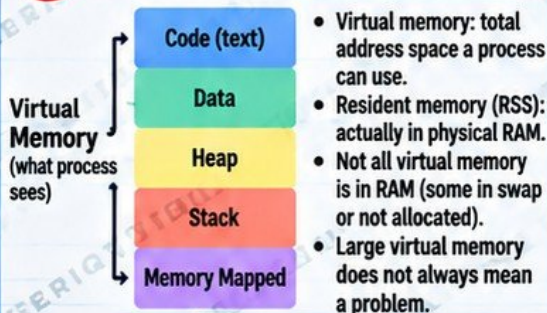
- Load average is high but CPU shows low %usage.
- High `iowait` or `steal` time.
- Many processes in uninterruptible sleep (D state).
- Could be disk I/O, network storage or virtualization.
- Check run queue, `iowait`, steal time and blocked processes.
- Investigate with: `top`, `mpstat`, `iostat`, `ps`, `dmesg`.

Memory Pressure, OOM and Swap Failures

Understand Linux memory behaviour, identify pressure and troubleshoot memory-related failures.

Real Systems
Real Problems
Real Solutions

1 VIRTUAL vs RESIDENT MEMORY



2 PAGE CACHE AND ANONYMOUS MEMORY

Page Cache



- Caches file data in memory.
- Improves I/O performance.
- Reclaimable when needed.

Anonymous Memory



- Used by applications (heap, stack).
- Not backed by files.
- Cannot be easily reclaimed.

3 SWAP BEHAVIOUR



RAM
(fast)



Swap
(slow)

- Used when RAM is low.
- Prevents immediate OOM.
- Too much swap usage degrades performance.
- Continuous swapping indicates memory pressure.

4 MAJOR AND MINOR PAGE FAULTS

- Minor page fault: page is already in memory (page table update).
- Major page fault: page must be loaded from disk (slow, increases latency).
- High major faults indicate memory pressure or insufficient RAM.

Check with `vmstat` or `/proc/vmstat`.

5 MEMORY PRESSURE

- System is running low on available memory.
- Kernel reclaims page cache and unused memory.
- Swap usage increases.
- Performance degrades (slow responses).
- If memory cannot be reclaimed, OOM killer may terminate processes.

Watch for increasing swap, high page faults and processes getting killed.

6 OOM KILLER DECISIONS

- Kernel kills processes when memory cannot be reclaimed.
- Chooses based on:
 - Memory usage (RSS)
 - oom_score and oom_score_adj
 - Process importance
 - cgroup limits
 - System state

Check `/var/log/kern.log` or `dmesg` for OOM killer messages.

7 USEFUL COMMANDS

```
free -h          # memory and swap usage
vmstat 1 5       # memory, swap, page faults
cat /proc/meminfo # detailed memory info
dmesg | grep -i oom # OOM killer logs
ps aux --sort=-%mem # processes by memory
pmap -x <pid>     # memory usage per process
```

8 MEMORY LEAKS

- Process memory usage keeps growing over time.
- Not released back to the OS.
- Leads to increased RSS and eventual OOM.
- Identify with:
 - ps, top, smem
 - Memory profiling tools (valgrind, heaptrack, eBPF tools)
 - Application metrics

9 CGROUP MEMORY LIMITS

- Containers and services can have memory limits using cgroups.
- If a process exceeds the limit:
 - It may be OOM killed inside the cgroup.
 - Host may still have free memory.
- Check limits and usage:

```
cat /sys/fs/cgroup/memory.max
cat /sys/fs/cgroup/memory.current
docker stats # for containers
kubectl describe pod <pod>
```

10 PRODUCTION SCENARIO: PROCESS DISAPPEARS WITHOUT APPLICATION ERROR

Symptoms



- Process suddenly disappears.
- No error in application logs.
- Service restarts or becomes unavailable.
- dmesg shows OOM killer message.

Investigation Steps



- Check `dmesg` or `/var/log/kern.log` for OOM.
- Review memory usage before the event.
- Check for memory leaks or sudden increase.
- Verify cgroup limits (if containerized).
- Identify what triggered the memory spike.

Resolution / Prevention



- Optimize memory usage.
- Set appropriate memory limits.
- Add monitoring and alerts.
- Fix memory leaks in the application.
- Plan capacity and load testing.



"A process disappearing without an application error is often the OOM killer, not a bug in your code."

Disk I/O Saturation and Storage Latency

Find the bottleneck. Measure. Verify. Fix the right thing.

Real Systems
Real Problems
Real Solutions

1 IOPS vs THROUGHPUT vs LATENCY

Metric	What it means	Typical unit	When it matters
IOPS	Number of I/O operations per second	IOPS	High IOPS needed for many small reads/writes
Throughput	Amount of data transferred per second	MB/s or GB/s	High throughput for large sequential reads/writes
Latency	Time taken to complete an I/O request	ms	High latency slows applications even if IOPS/throughput looks fine

Applications need a balance of IOPS, throughput and low latency.

2 QUEUE DEPTH



- Queue depth = number of I/O requests waiting to be processed.
- High queue depth indicates the device is saturated.
- Consistently high queue depth leads to increased latency.
- Check with `iostat` (`aqu-sz`) or cloud monitoring tools.

If queue depth keeps growing, your storage cannot keep up with the demand.

3 I/O WAIT



- I/O wait is the time CPUs spend waiting for disk I/O to complete.
- High `iowait` means the system is bottlenecked by storage.
- Check with `top`, `vmstat` or `mpstat`.
- Sustained `iowait` (> 10-20%) usually indicates storage issues.

High CPU `iowait` = CPUs are idle, waiting for disk.

4 DEVICE UTILIZATION



- Device utilization shows how busy the storage device is (0% to 100%).
- Near 100% utilization indicates saturation.
- Check with `iostat` (%util).
- Look at both read and write utilization.

Sustained 100% utilization = storage is the bottleneck.

5 READ / WRITE LATENCY



Read Latency



Write Latency

- Check average read and write latency (in milliseconds).
- High latency directly affects application performance.
- Use `iostat`, cloud monitoring or storage vendor tools.

Even with low utilization, high latency can still cause application slowdowns.

6 USEFUL COMMANDS

```
$ iostat -x 15 # disk I/O statistics
$ iotop # real-time I/O by process
$ pidstat -d 15 # per-process I/O stats
$ vmstat 15 # system I/O wait
$ top # overview (check iowait)
$ dmesg | grep -i sd # disk-related kernel logs
```

Use multiple tools to get a complete picture.

7 IDENTIFYING THE PROCESS GENERATING I/O



- Use `iotop` to see real-time I/O by process.
- Use `pidstat -d` to check per-process I/O.
- Correlate high I/O processes with application behaviour.
- Look for unexpected processes or runaway logs.

Find the actual process before taking action.

8 FILESYSTEM vs BLOCK-DEVICE PROBLEMS



Filesystem issues

- High metadata operations (inodes).
- Filesystem corruption.
- Journal or writeback delays.
- Check with `df`, `dmesg`, `fsck` and `mount` options.

Block-device issues

- High latency or I/O errors.
- Hardware or cloud volume limits.
- Check with `iostat`, `dmesg` and `smartctl` (if available).

Determine if the problem is at the filesystem layer or the underlying storage device.

9 CLOUD-VOLUME PERFORMANCE LIMITS



- Cloud volumes have defined IOPS, throughput and latency limits.
- Exceeding limits leads to throttling and increased latency.
- Examples: AWS EBS, Azure Disk, GCP Persistent Disk.
- Check your instance type and volume performance settings.
- Monitor with cloud provider metrics (CloudWatch, Azure Monitor, etc).

Know the limits of your cloud storage. Scale when needed.

10 PRODUCTION SCENARIO



Scenario: Application latency caused by storage saturation.

- Symptoms: slow response times, high `iowait`, high disk latency.
- Investigation: check `iostat`, identify high I/O process with `iotop`.
- Root cause: storage volume at 100% utilization with high queue depth.
- Fix: optimize I/O, increase volume size/IOPS, or move to a faster storage type.
- Prevention: monitor disk metrics and set alerts.

High application latency is often a storage issue, not a CPU issue.

Filesystem Capacity, Inodes and Mount Failures

Diagnose filesystem issues quickly. Find the real cause. Apply the right fix.

Real Systems
Real Problems
Real Solutions

1 DISK SPACE vs INODE EXHAUSTION



Disk space = amount of storage (GB).
Inodes = number of files/directories.

- You can have free disk space but no inodes.
- When inodes are exhausted, you cannot create new files even if `df -h` shows space available.
- Common on systems with many small files (logs, caches, temp files).

Always check both disk space and inodes.

2 df -h vs df -i

```
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1      100G   70G   30G   70% /
/dev/sdb1       500G  200G  300G   40% /data
```

```
$ df -i
Filesystem      Inodes   IUsed   IFree  IUse% Mounted on
/dev/sda1    6553600 6553000    600    99% /
/dev/sdb1    32768000 100000 32668000    1% /data
```

Use `df -h` to check space.
Use `df -i` to check inodes.

3 DELETED BUT OPEN FILES



- A file can be deleted but still open by a process.
- The space is not released until the process closes it.
- Common with log files.

```
$ ls -l +L1
COMMAND PID USER FD TYPE SIZE/OFF NODE NAME
java 1234 app 4w REG 2.1G 12345 /var/log/app.log (deleted)
nginx 5678 root 7w REG 500M 67890 /var/log/nginx.log (deleted)
```

Find and restart the process or truncate the file descriptor.

4 MOUNT FAILURES



Mount can fail due to:

- Wrong device name
- Missing filesystem
- Incorrect mount options
- Network/storage issues (NFS, EFS, etc.)
- Permission problems

```
$ mount /dev/sdb1 /data
mount: /data: wrong fs type, bad option,
bad superblock...
```

Check the error message. It usually tells you the real cause.

5 READ-ONLY REMOUNTS



Filesystems can be remounted read-only when:

- Filesystem errors are detected
- Underlying storage issues
- Power failures or unsafe shutdowns
- Kernel protection (to prevent further damage)

```
$ mount | grep ' /'
/dev/sda1 on / type ext4 (ro,relatime)
```

Check `dmesg` and logs to find out why it was remounted read-only.

6 FILESYSTEM CORRUPTION INDICATORS



Common signs:

- I/O errors in `dmesg`
- Filesystem inconsistencies
- Unexpected read-only mounts
- Slow performance
- Applications failing to read/write

```
$ dmesg | grep -iE 'ext4|xfs|error|fail'
[ 1234.567] EXT4-fs error (device sda1):
ext4_journal_check_start:56: Detected
journal corruption
[ 1234.568] Buffer I/O error on dev sda1
```

Run filesystem checks during maintenance (e.g. `fsck` for ext4, `xfs_repair` for XFS).

7 /etc/fstab



- Defines filesystems to be mounted at boot.
- Wrong entries can cause boot delays or failures.
- Check device names, mount options and filesystem types.

```
$ cat /etc/fstab
/dev/sda1 / ext4 defaults 0 1
/dev/sdb1 /data xfs defaults 0 2
UUID=xxx /backup ext4 defaults 0 2
```

Use UUID instead of `/dev/sdX` for stability.

8 TEMPORARY FILESYSTEM PRESSURE



- `/tmp`, `/var/tmp` and other temporary filesystems can fill up.
- Often caused by:
 - Application temp files
 - Large package installations
 - Logs or core dumps
 - Container workloads

```
$ df -h /tmp
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           4.0G  3.8G  200M   95% /tmp
```

Monitor and clean up temporary files regularly.

9 PRODUCTION SCENARIO

"Disk full" when `df` appears normal



Symptoms:

- Cannot create new files.
- Applications start failing.
- `df -h` shows plenty of space.
- Often caused by inode exhaustion or deleted but open files.

Investigation Steps:

- Run `df -i` to check inode usage.
- Run `ls -l +L1` to find deleted open files.
- Check logs and `dmesg` for filesystem errors.
- Identify the process causing the issue.
- Clean up or restart the process.
- Implement monitoring to prevent recurrence.

Don't trust just `df -h`. Always check the bigger picture.

Process, Thread and Zombie Investigation

Understand how processes work. Find issues fast. Keep your systems healthy.

Real Systems
Real Problems
Real Solutions

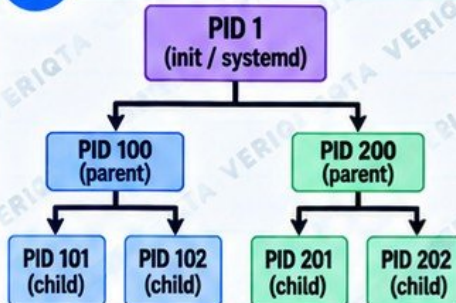
1 PROCESS LIFECYCLE



- A process is created using `fork()`.
- `exec()` loads a new program into the process.
- The process runs, may wait (sleep) for resources, and eventually exits.
- The kernel manages the process lifecycle.

Every process goes through a lifecycle.
Issues can happen at any stage.

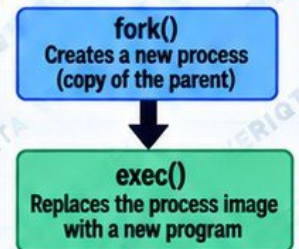
2 PID, PPID AND PROCESS TREES



- PID = Process ID (unique number).
- PPID = Parent Process ID.
- Processes form a tree (parent → child).
- Use `ps` or `ps` to view the tree.

Understanding the process tree helps
you find the root cause.

3 FORK AND EXEC



- `fork()` creates a new process with the same memory space.
- `exec()` replaces the process with a new program.
- Common in shells, web servers and service managers.

Fork creates. Exec replaces.
Together they run new programs.

4 PROCESS STATES

State	Code	Description
Running	R	Actively using CPU
Sleeping	S	Waiting for an event
Uninterruptible	D	Waiting for I/O (cannot be killed)
Stopped	T	Stopped (e.g. by signal)
Zombie	Z	Process finished but not reaped

Check process states with `ps`, `top` or `/proc`.

Many problems are caused by processes stuck
in D (uninterruptible) state.

5 SLEEPING AND UNINTERRUPTIBLE PROCESSES



- Sleeping (S): waiting for an event (e.g. I/O, timer).
- Uninterruptible (D): waiting for I/O at the kernel level.
- D state processes cannot be killed easily.
- Often caused by disk, network or filesystem issues.

```
$ ps -eo pid,stat,comm | grep ' D '
$ cat /proc/<pid>/stack
```

Many D state processes indicate a
storage or filesystem problem.

6 ZOMBIES AND ORPHANED PROCESSES



- Zombie (Z): process has finished but the parent has not reaped it.
- Uses a PID but no resources (usually).
- Orphaned: parent has exited, child is adopted by PID 1.
- Too many zombies can exhaust the PID limit.

```
$ ps -eo pid,ppid,stat,cmd | grep ' Z '
$ pstree -p
```

Zombies are usually a sign of a problem
in the parent process.

7 THREAD EXPLOSION



- A process can have many threads.
- Too many threads can exhaust system resources (CPU, memory, PIDs).
- Common causes: application bugs, misconfiguration, or load spikes.
- Check thread count with `ps -L` or `top -H`.

```
$ ps -eLf | grep <pid>
$ top -H -p <pid>
```

Unexpected high thread count can
slow down or crash the system.

8 USEFUL COMMANDS

```
ps aux          # list all processes
ps -eo pid,ppid,stat,cmd # custom output
pstree -p       # process tree
top             # real-time view
pidstat 1       # per-process stats
cat /proc/<pid>/status # process details
cat /proc/<pid>/stack # kernel stack (D state)
ls /proc/<pid>/fd # open files
```

Use multiple tools to get a complete picture.
Correlate with metrics and logs.

9 PRODUCTION SCENARIO

Thousands of processes exhaust
system resources

Symptoms:

- System becomes slow or unresponsive.
- Cannot create new processes (fork: Resource temporarily unavailable).
- High load average.
- Services fail to start.
- May be caused by a fork bomb, application bug or runaway script.

Investigation Steps:

- 1 Check process and thread count.
- 2 Identify the parent process.
- 3 Find what is creating the processes.
- 4 Stop or limit the process.
- 5 Implement monitoring and alerts.

Signals, Hung Processes and Safe Termination

Real Systems
Real Problems
Real Solutions

Understand how process signals work. Terminate safely. Protect your systems and data.

1 SIGTERM vs SIGKILL

SIGTERM (15)

- Default termination signal.
- Allows the process to clean up.
- Can be trapped and ignored by the application.
- Used for graceful shutdown.

SIGKILL (9)

- Forcibly terminates the process.
- Cannot be caught, blocked or ignored.
- Does not allow cleanup.
- Use only when SIGTERM fails.

Always try SIGTERM first.
Use SIGKILL only as a last resort.

2 SIGNAL HANDLING

- Processes can catch, ignore or handle signals.
- Applications define custom signal handlers.
- Some processes (e.g. Java, Nginx, PostgreSQL) perform cleanup on SIGTERM.
- If a process ignores SIGTERM, it may continue running.

```
$ trap -l          # list signals
$ kill -l          # show signal list
$ trap 'echo "Terminated"' TERM
```

Proper signal handling allows graceful shutdown and prevents data loss.

3 GRACEFUL SHUTDOWN



- Allows the application to finish ongoing work.
- Closes connections and flushes data.
- Releases resources cleanly.
- Common in web servers, databases and message queues.

```
$ kill -TERM <pid>          # graceful stop
$ systemctl stop <service> # stop service
```

Graceful shutdown keeps your system stable and your data safe.

4 PROCESSES STUCK IN D STATE

- D state (uninterruptible sleep) means the process is waiting for I/O (e.g. disk, network, filesystem).
- Cannot be killed with SIGKILL.
- Common causes:
 - Unresponsive storage
 - NFS or network filesystem issues
 - Hardware or driver problems

```
$ ps -eo pid,stat,cmd | grep ' D '
$ cat /proc/<pid>/stack
$ dmesg | tail
```

If a process is in D state, fix the underlying I/O issue.
The process cannot be killed.

5 WHY KILL -9 SOMETIMES CANNOT KILL A PROCESS

- SIGKILL cannot be caught, but it can still fail when:
 - Process is in D state (waiting for I/O).
 - Kernel is blocked.
 - Filesystem or storage is unresponsive.
 - It is a kernel thread.
 - Zombie process (already exited).



If kill -9 does not work, the process is usually in D state or the system is having a deeper kernel or storage issue.

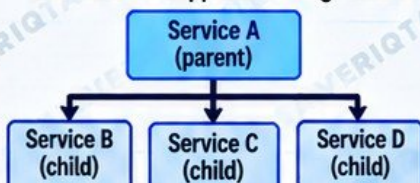
6 USEFUL COMMANDS

```
$ kill -TERM <pid>          # graceful stop
$ kill -KILL <pid>          # force stop
$ pkill <name>              # kill by process name
$ pgrep <name>              # find PIDs by name
$ ps -eo pid,stat,cmd       # check process state
$ strace -p <pid>           # see system calls
$ lsof -p <pid>             # open files
$ cat /proc/<pid>/status     # process details
$ cat /proc/<pid>/stack      # kernel stack (D state)
```

Use these tools together to understand why a process will not terminate.

7 SHUTDOWN DEPENDENCIES

- A process may rely on other services, files or network resources.
- Child processes may keep the parent alive.
- Open file descriptors can prevent shutdown.
- Systemd services have dependencies that must be stopped in the right order.



Identify and stop dependencies first to avoid hanging processes.

8 DATA CORRUPTION RISK

- Forcing termination can leave data in an inconsistent state.
- Risk is higher for databases, filesystems and applications writing to disk.
- Always prefer graceful shutdown.
- Verify data integrity after a forced kill.



Killing a process may solve the immediate problem but can lead to data corruption.

9 PRODUCTION SCENARIO

Service refuses to terminate during deployment

Symptoms:

- Service does not stop after kill -TERM.
- kill -9 has no effect.
- Deployment hangs or times out.
- Process is in D state or waiting for resources.
- May cause failed deployments and downtime.

Investigation Steps:

- 1 Check process state with ps.
- 2 Inspect logs and system resources.
- 3 Use strace to see what the process is waiting for.
- 4 Identify and resolve underlying I/O or dependency issue.
- 5 Retry graceful shutdown.
- 6 Only use SIGKILL if safe and necessary.

Fix the root cause, not just the symptom.

Understand how systemd works. Diagnose service failures. Keep your systems running.

1 UNIT DEPENDENCIES



- Services can depend on other units.
- Dependencies ensure services start in the correct order.
- Common dependency directives:

- Requires=
- Wants=
- After=
- Before=

Check dependencies when a service fails to start.

2 ORDERING

- Ordering defines the sequence in which units start.
- Common directives:
 - After= (start after the specified unit)
 - Before= (start before the specified unit)

```
$ systemctl list-dependencies myapp.service
```



Use correct ordering to avoid race conditions during startup.

3 TARGETS

- Targets are groups of units.
- Common targets:
 - multi-user.target (normal boot)
 - graphical.target (GUI)
 - network.target (network services)
 - basic.target (minimal system)
 - rescue.target (recovery mode)

- Check which target a service is part of.

```
$ systemctl get-default
$ systemctl list-units --type=target
```

A service may fail if the required target is not reached.

4 RESTART POLICIES

- systemd can automatically restart failed services.
- Common restart settings:

- Restart=no (default)
- Restart=on-failure
- Restart=always
- Restart=on-abnormal
- RestartSec=5 (delay before restart)



Use restart policies carefully. Fix the root cause, not just restart.

5 EXIT CODES

- Every service exits with a code.
- Exit code 0 = success.
- Non-zero codes indicate failure.
- Common causes:

- 1 General error
- 2 Misuse of shell builtins
- 126 Permission denied
- 127 Command not found
- >128 Killed by a signal

Check the exit code in logs to find the real cause.

6 USEFUL COMMANDS

```
$ systemctl status myapp # service status
$ systemctl start myapp # start service
$ systemctl stop myapp # stop service
$ systemctl restart myapp # restart service
$ systemctl enable myapp # enable at boot
$ systemctl disable myapp # disable at boot
$ systemctl list-dependencies myapp
$ journalctl -u myapp # view logs
$ journalctl -u myapp -f # follow logs
$ systemctl show myapp # show unit details
```

Logs tell the real story. Always check journalctl.

7 FAILED ENVIRONMENT / CONFIGURATION

- Common causes:
 - Missing environment variables
 - Wrong file paths
 - Incorrect permissions
 - Invalid configuration files
 - Different environment when run by systemd (not your user)



Check the environment and configuration used by systemd.

8 RESTART LOOPS

- A service keeps failing and restarting.
- Causes:

- Configuration errors
- Missing dependencies
- Permission issues
- Application crashes
- Incorrect environment

```
$ systemctl status myapp
$ journalctl -u myapp -n 50
$ systemctl reset-failed myapp
```

A restart loop is a symptom. Find and fix the root cause.

9 PRODUCTION SCENARIO

Service works manually but fails under systemd.

Symptoms:

- Works when run manually as a user.
- Fails to start with systemd.
- Application may exit immediately.
- No obvious error in the application logs.

Investigation Steps:

- 1 Check journalctl -u <service>.
- 2 Compare environment variables.
- 3 Verify working directory and file paths.
- 4 Check permissions (user/group).
- 5 Review unit file (ExecStart, Environment, etc).
- 6 Fix configuration and restart the service.

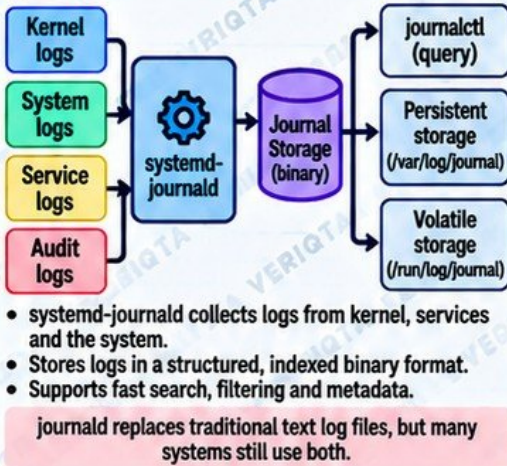
Same command, different environment. Check what systemd is actually running.

Linux Logging and journald Investigation

Find the evidence. Correlate the events. Solve the real problem.

Real Systems
Real Problems
Real Solutions

1 SYSTEMD JOURNAL ARCHITECTURE



2 KERNEL vs SERVICE LOGS

Kernel Logs	Service Logs
- Hardware events - Driver issues - Kernel errors/warnings - System crashes - Available with -k flag	- Application logs - Service-specific events - Startup and shutdown - Errors and exceptions - Use -u <service>

```

$ journalctl -k # kernel logs
$ journalctl -u nginx # nginx service logs
$ journalctl -u sshd # sshd service logs
    
```

Kernel logs help you find system-level issues.
Service logs show application behaviour.

3 BOOT-SPECIFIC INVESTIGATION

- Useful after a crash, reboot or failed startup.
- Each boot has a unique boot ID.
- Use **-b** to view logs for the current or previous boots.

```

$ journalctl -b # current boot
$ journalctl -b -1 # previous boot
$ journalctl --list-boots # list all boots
    
```

Example output (journalctl --list-boots):

```

-1 a3f2... Mon 2024-06-10 10:15:22 UTC
0 b7c1... Tue 2024-06-11 08:03:11 UTC
    
```

Boot-specific logs help you investigate issues that happened during startup.

4 TIME-WINDOW CORRELATION

- Correlate events across services and the kernel.
- Use time filters to focus on the relevant period.
- Combine logs from multiple sources.



```

$ journalctl --since "2024-06-10 10:00" \
--until "2024-06-10 12:00"
$ journalctl -u nginx --since "1 hour ago"
$ journalctl -k --since "today"
    
```

Time correlation helps you build the incident timeline and find the root cause.

5 USEFUL journalctl COMMANDS

```

$ journalctl -u <service> # service logs
$ journalctl -b # current boot
$ journalctl -b -1 # previous boot
$ journalctl -k # kernel logs
$ journalctl --since "1h ago"
$ journalctl --until "now"
$ journalctl -p err # only errors
$ journalctl -f # follow live
$ journalctl --no-pager # clean output
    
```

Use these options to quickly find the information you need.

6 LOG ROTATION

- journald manages its own rotation.
- Controlled by `/etc/systemd/journald.conf`.
- Logs are rotated based on size, time and retention settings.



Key settings:

- SystemMaxUse=
- SystemKeepFree=
- MaxRetentionSec=
- MaxFileSec=

```

$ cat /etc/systemd/journald.conf | grep -v ^#
$ journalctl --disk-usage
$ journalctl --vacuum-time=7d # clean old logs
    
```

Proper log rotation prevents disk from filling up.

7 MISSING LOGS

- Logs may be missing due to:
 - Short retention period
 - Logs rotated or vacuumed
 - Service logs not going to journald
 - Misconfigured logging (e.g. stdout)
 - Disk space issues
 - Wrong time or timezone



If logs are missing, check configuration, retention settings and disk space.

8 DISK PRESSURE FROM LOGGING

- Logs can consume significant disk space.
- Large or noisy services can fill `/var/log/journal`.
- Check disk usage and set limits.



```

$ journalctl --disk-usage
$ du -sh /var/log/journal
$ df -h
$ journalctl --vacuum-size=1G
    
```

Monitor log size and set appropriate retention limits.

9 TIME SYNCHRONIZATION IMPLICATIONS

- Accurate time is critical for log analysis.
- Use NTP or chrony to keep time in sync.
- Wrong time leads to incorrect timelines and correlation.



```

$ timedatectl # check time status
$ chronyc sources # if using chrony
$ ntpq -p # if using ntpd
    
```

Always ensure correct time and timezone on all systems.

10 PRODUCTION SCENARIO: RECONSTRUCTING AN INCIDENT FROM MULTIPLE EVIDENCE SOURCES



Scenario:

- A service was down for 20 minutes.
- Need to reconstruct what happened using kernel logs, service logs and system logs.
- Correlate events across the correct time window.
- Identify the root cause and verify the fix.

Investigation Steps:

- Check the timeline (time and timezone).
- Review boot logs if there was a reboot.
- Analyze service logs (`journalctl -u`).
- Check kernel logs (`journalctl -k`).
- Correlate events across services.
- Identify the root cause.
- Implement a fix and monitor.

Key Takeaway:

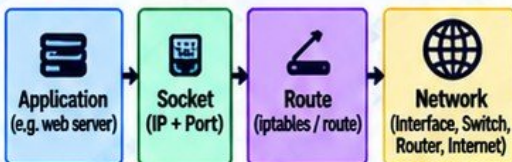
Use all available evidence (kernel logs, service logs, system logs, time correlation) to build a complete picture of the incident.

Network Connectivity Failure Isolation

Follow the path. Find the break. Restore connectivity.

Real Systems
Real Problems
Real Solutions

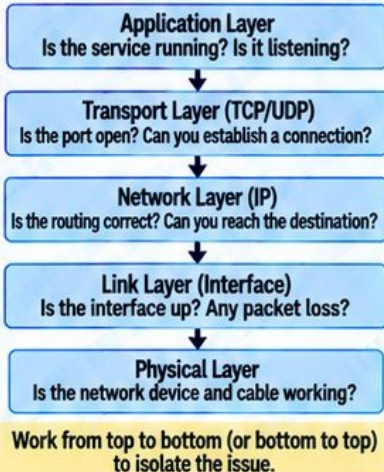
1 APPLICATION TO SOCKET TO ROUTE TO NETWORK



- Application uses a socket to send/receive data.
- The OS determines the route to the destination.
- Traffic goes through network interfaces, firewall rules and routers to reach the target.

A failure at any layer can prevent connectivity.

2 LAYERED TROUBLESHOOTING



3 USEFUL COMMANDS

```
$ ip addr # show network interfaces
$ ip route # show routing table
$ ss -tlnp # show listening ports
$ ping <host> # test basic connectivity
$ traceroute <host> # trace the network path
$ nc -zv <host> <port> # check port reachability
$ curl -v <url> # test HTTP/HTTPS
$ dig <domain> # DNS lookup
$ nslookup <domain> # DNS lookup (alternative)
```

These commands help you identify where the connection is failing.

4 ROUTING FAILURES

- Incorrect or missing routes.
- Wrong default gateway.
- Asymmetric routing.
- Blackhole routes.
- Misconfigured static or dynamic routing (e.g. BGP, OSPF).
- Cloud route table issues (e.g. AWS, Azure).



```
$ ip route show
$ ip route get <destination-ip>
$ traceroute <destination-ip>
```

If the route is wrong, packets may never reach the destination.

5 INTERFACE STATE

- Interface is down or administratively down.
- Link issues (no carrier).
- Speed/duplex mismatches.
- Errors and dropped packets.



```
$ ip link show
$ ip addr show
$ ethtool <interface>
$ dmesg | grep -i <interface>
```

An interface must be UP and have an IP configuration to send and receive traffic.

6 PORT REACHABILITY

- Check if the service is listening on the expected port.
- Verify local and remote reachability.
- Consider firewall rules and security groups.

```
$ ss -tlnp | grep <port>
$ nc -zv <host> <port>
$ telnet <host> <port>
$ curl -v http://<host>:<port>
```



A port can be open locally but still blocked by a firewall or network boundary.

7 FIREWALL BOUNDARIES



- Host firewall (iptables / nftables).
- Cloud security groups (AWS, Azure, GCP).
- Network ACLs.
- Intermediate firewalls or load balancers.

```
$ sudo iptables -L -n
$ sudo nft list ruleset
$ sudo ufw status
$ cloud firewall / security group rules
```

Even if the service is running, firewall rules can block external access.

8 PACKET LOSS

- Network congestion.
- Unreliable links.
- Dropped packets due to firewall or rate limiting.
- Check latency and loss to identify issues.

```
$ ping <host>
$ mtr <host>
$ traceroute <host>
$ tc -s qdisc # check traffic stats
```

High packet loss can cause timeouts and intermittent connectivity problems.

9 PRODUCTION SCENARIO

Service reachable locally but unreachable remotely



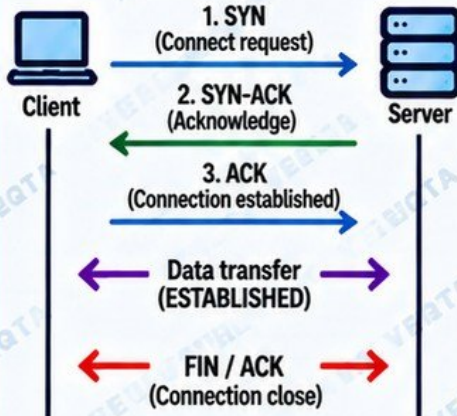
Possible Causes:

- Firewall blocking external access.
- Service listening only on localhost (127.0.0.1).
- Missing or incorrect routing.
- Cloud security group rule missing.
- Load balancer or proxy misconfiguration.
- Interface or network ACL issue.
- Asymmetric routing.

Always test from both local and remote locations to identify where the connection fails.

1 TCP LIFECYCLE

TCP uses a three-way handshake to establish a connection, then keeps the connection open for data transfer, and finally closes it.



TCP is a stateful protocol. Each connection has a state on both client and server.

2 TCP STATES

Common TCP connection states and what they mean:

State	Meaning
SYN_SENT	Client has sent SYN and is waiting for SYN-ACK.
SYN_RECV	Server has received SYN and is waiting for ACK.
ESTABLISHED	Connection is open and data can be transferred.
FIN_WAIT_1	A side has sent FIN and is waiting for ACK.
FIN_WAIT_2	A side has received ACK for FIN and is waiting for the other side.
CLOSE_WAIT	A side has received FIN and is waiting for the application to close.
LAST_ACK	A side has sent FIN and is waiting for the final ACK.
TIME_WAIT	Connection closed, waiting to ensure old packets do not interfere with new connections.

Stuck connections in certain states can indicate problems on the client, server or network.

3 USEFUL COMMANDS

```

$ ss -s # socket summary
$ ss -tanp # TCP connections with PIDs
$ ss -state syn-sent # show SYN_SENT
$ ss -state syn-recv # show SYN_RECV
$ ss -state established # show ESTABLISHED
$ ss -state time-wait # show TIME_WAIT
$ ss -state close-wait # show CLOSE_WAIT
$ netstat -s # protocol statistics
$ cat /proc/net/sockstat # socket usage
$ cat /proc/sys/net/ipv4/ip_local_port_range # ephemeral port range
  
```

Other useful tools:

- tcpdump # capture packets
- iperf3 # test network performance
- nmap # check if port is reachable
- lsof # list open sockets
- sar # network statistics

Use these commands to understand connection states, find bottlenecks and identify failures.

4 CONNECTION BACKLOG

- When a client connects, it is placed in a queue before the application accepts it.
- Two queues:
 - SYN backlog (half-open connections)
 - Accept backlog (fully established, waiting for accept())
- If the queues are full, new connections may be dropped or reset.

```

$ cat /proc/sys/net/ipv4/tcp_max_syn_backlog
$ cat /proc/sys/net/core/somaxconn
  
```

Increase backlog values if you expect high traffic and are seeing connection drops.

5 SOCKET EXHAUSTION

- Each TCP connection uses a socket and system resources (memory, file descriptors).
- Symptoms:
 - Cannot create new connections
 - "Too many open files"
 - High number of sockets in TIME_WAIT
- Check limits:

```

$ ulimit -n # open file limit
$ cat /proc/sys/fs/file-max # system limit
$ ss -s # socket usage summary
  
```

Increase limits and fix the underlying issue causing too many open connections.

6 EPHEMERAL PORTS

- Clients use ephemeral ports (temporary source ports) to make outbound connections.
- If ports are exhausted, new connections fail.
- Check the port range:

```
$ cat /proc/sys/net/ipv4/ip_local_port_range
```

Example output: 32768 60999

- You can change it (as root):

```
$ sysctl -w net.ipv4.ip_local_port_range="1024 65335"
```

Increase the ephemeral port range if you have many outbound connections.

7 RETRANSMISSIONS

- Retransmissions happen when packets are lost or acknowledgments are not received.
- High retransmissions indicate:
 - Network congestion
 - Packet loss
 - Firewall or load balancer issues
 - Slow or overloaded server

```

$ ss -ti <connection> # show TCP details
$ sar -n DEV 1 # check network errors
  
```

Frequent retransmissions can lead to high latency and connection timeouts.

8 PRODUCTION SCENARIO

Connection failures during traffic spikes

Symptoms:

- Users see timeouts or connection refused.
- Application logs show failing to connect.
- High number of SYN_SENT or TIME_WAIT states.
- New connections fail even though the service is running.

Possible causes:

- Connection backlog is full.
- Socket or file descriptor exhaustion.
- Ephemeral ports exhausted.
- Network packet loss or firewall limits.
- Server overloaded and not accepting connections.

9 INVESTIGATION STEPS

- 1 Check service status and system resources.
- 2 Use ss -s and ss -tanp to review connection states.
- 3 Look for high SYN_SENT, TIME_WAIT or CLOSE_WAIT.
- 4 Check backlog, file descriptor and ephemeral port limits.
- 5 Capture packets with tcpdump if needed.
- 6 Verify network devices (firewall, load balancer, security groups).
- 7 Tune limits and fix the root cause.

Follow a systematic approach. Check each layer from application to network to isolate the issue.

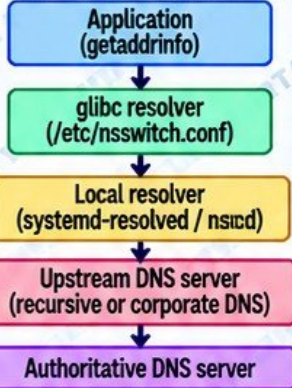
DNS Resolution Failures

Real Systems
Real Problems
Real Solutions

Understand how DNS works. Find the issue. Keep your services running.

1 RESOLVER PATH

A DNS query follows a path from the application to a resolver, then to upstream DNS servers.



A failure at any point in the path can break name resolution.

2 /etc/resolv.conf

- Defines the DNS servers and search domains used by the system.
- Can be managed by systemd-resolved, NetworkManager or manually.

```
$ cat /etc/resolv.conf
nameserver 10.0.0.2
nameserver 8.8.8.8
search example.com
options ndots:5 timeout:2 attempts:3
```

Key settings:

- nameserver** – DNS server IP
- search** – search domain(s)
- options** – timeout and retry behaviour

Incorrect DNS servers or options can cause slow or failed resolution.

3 /etc/nsswitch.conf

- Controls how the system resolves names (hosts, users, groups, etc).
- The hosts line defines the order of DNS resolution.

```
$ cat /etc/nsswitch.conf
hosts:      files dns
# Possible values: files, dns, mdns4_minimal
# [NOTFOUND=return] ...
```

Common configurations:

- files dns** (check /etc/hosts first, then DNS)
- dns files** (use DNS first)
- myhostname dns** (systemd-resolved)

Wrong order or misconfiguration can cause unexpected resolution failures.

4 SEARCH DOMAINS

- Search domains are appended automatically when a hostname is not fully qualified.
- Can cause unexpected queries or delays.

```
$ cat /etc/resolv.conf
search example.com corp.local
```

Example:

- Query:** app
- Search:** example.com, corp.local
- Tries:** app.example.com
app.corp.local
app (if ndots allows)

Search domains are useful but can also cause delays if misconfigured.

5 RESOLVER CACHING

- DNS responses are cached to improve performance.
- Caching can be done by:
 - systemd-resolved
 - nsd (Name Service Cache Daemon)
 - local DNS servers (e.g. dnsmasq)

```
# Check systemd-resolved status
$ resolvectl status
# Flush the cache
$ resolvectl flush-caches
```

Useful commands:

- resolvectl status** # check cache and servers
- systemd-resolve --statistics** # cache stats

Stale or incorrect cache entries can cause intermittent resolution failures.

6 USEFUL COMMANDS

```
$ dig example.com           # detailed DNS query
$ dig +short example.com    # short output
$ getent hosts example.com   # system resolver
$ resolvectl query example.com # systemd resolver
$ nslookup example.com      # alternative tool
$ host example.com           # simple query
$ ping example.com           # test resolution
$ traceroute example.com     # check network path
```

Check which DNS server is being used:

```
$ resolvectl status
```

Check all queries in real time:

```
$ sudo journalctl -u systemd-resolved -f
```

These tools help you test, diagnose and compare DNS resolution behaviour.

7 NXDOMAIN vs TIMEOUT

Result	Meaning	Common Cause
NXDOMAIN	Domain does not exist	Wrong domain name, missing record
TIMEOUT	No response received	Network issue, firewall block, DNS server down
SERVFAIL	Server failed to resolve	DNS server misconfig or upstream issue
NOERROR (empty)	Valid response but no record	No A/AAAA record for the queried type

Different errors require different troubleshooting approaches.

8 DNS LATENCY

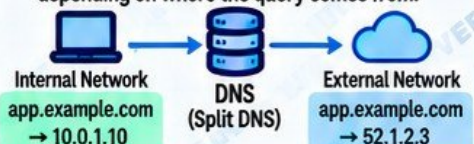
- Slow DNS responses can cause application timeouts.
- Causes of high latency:
 - overloaded DNS server
 - long search domain list
 - network latency to DNS server
 - retries due to packet loss

```
# Measure DNS query time
$ dig example.com
# Check resolver performance
$ resolvectl statistics
```

High DNS latency can make applications appear slow or unavailable.

9 SPLIT DNS

- Different DNS responses based on network location (internal vs external).
- Common in corporate and cloud environments.
- The same domain can resolve to different IPs depending on where the query comes from.



Production Scenario

- Application failure despite healthy network connectivity.
- Application cannot resolve required domain.
- Network, routing and firewall are working.
- Caused by DNS misconfiguration, split DNS or resolver issue.

1 WHEN PACKET CAPTURE IS JUSTIFIED

Use packet capture when:

- You cannot find the issue from logs alone.
- You need to see actual network traffic.
- There is a mismatch between client and server behaviour.
- You need to prove whether the server or network is dropping traffic.
- You are investigating intermittent issues.
- You need to inspect protocol-level details (TCP, DNS, TLS, etc.).



Packet capture shows what is really happening on the wire.

2 INTERFACE SELECTION

- Choose the correct network interface for capture.
- Common interfaces:
 - eth0, eth1 (physical)
 - ens3, enp0s3 (modern naming)
 - any (all interfaces)
 - lo (loopback)



```
$ ip link show # list interfaces
$ tcpdump -i eth0 # specific interface
$ tcpdump -i any # capture on all
```

Capturing on the wrong interface will miss the traffic.

3 HOST AND PORT FILTERS

Filter packets to capture only what you need.

```
$ tcpdump host 192.168.1.10
$ tcpdump port 80
$ tcpdump host 10.0.0.5 and port 443
$ tcpdump src 10.0.0.5
$ tcpdump dst 192.168.1.20
$ tcpdump tcp port 22 and host 10.0.0.10
```



Filters reduce noise and make analysis easier.

4 TCP HANDSHAKE ANALYSIS

- A successful TCP connection follows a three-way handshake:



- Use tcpdump to verify if all three packets are present.
- Missing SYN-ACK may indicate firewall or server issue.
- Retransmissions indicate packet loss or high latency.

```
$ tcpdump -i eth0 tcp port 80 -nn -vv
```

Analyzing the handshake helps you identify where the connection fails.

5 RETRANSMISSIONS AND RESETS

- Retransmissions occur when packets are not acknowledged.
- Frequent retransmissions indicate network issues or server overload.
- TCP RST (reset) immediately terminates a connection.
- Check for unexpected RST packets.



```
$ tcpdump -i eth0 tcp -nn -vv | grep -E 'retransmission|RST'
```

Retransmissions and resets help identify network instability, firewall rules or application issues.

6 DNS PACKET INSPECTION

- Use tcpdump to inspect DNS queries and responses.
- Check for correct queries, responses, errors (NXDOMAIN), and response time.

```
$ tcpdump -i eth0 udp port 53 -nn -vv
```

Look for:

- DNS queries (A, AAAA, CNAME, etc.)
- Responses (success or NXDOMAIN)
- Slow responses (high latency)
- Incorrect or unexpected queries



Packet capture shows exactly what DNS traffic is going over the network.

7 TLS HANDSHAKE CLUES

- Use tcpdump to inspect TLS handshakes and identify common issues.

Look for:

- Client Hello
- Server Hello
- Certificate exchange
- TLS version and cipher suite
- Handshake failures (alerts)



```
$ tcpdump -i eth0 tcp port 443 -nn -vv
```

TLS handshake analysis can reveal certificate issues, protocol mismatches or handshake failures.

8 TCPDUMP COMMAND REFERENCE

Purpose	Command
Basic capture	<code>tcpdump -i eth0</code>
Capture to file	<code>tcpdump -i eth0 -w capture.pcap</code>
Read capture file	<code>tcpdump -r capture.pcap</code>
Capture with filter	<code>tcpdump -i eth0 port 80</code>
Verbose output	<code>tcpdump -i eth0 -nn -vv</code>
Limit capture size	<code>tcpdump -i eth0 -C 100 -W 5</code>
Capture DNS	<code>tcpdump -i eth0 udp port 53</code>
Capture TCP only	<code>tcpdump -i eth0 tcp</code>
Capture both sides	<code>tcpdump -i any host 10.0.0.5</code>

Use -w to save captures and analyze later with Wireshark if needed.

9 CAPTURE SAFETY AND SENSITIVE DATA

- Packet captures can contain sensitive information such as:

- Username and passwords (in plaintext protocols)
- Tokens and API keys
- Personal data
- Application payloads



- Follow best practices:
 - Capture only what you need
 - Use filters
 - Secure capture files
 - Sanitize before sharing
 - Follow your organization's security policies

Always handle packet captures as sensitive data.

10 PRODUCTION SCENARIO: PROVING WHETHER THE SERVER OR NETWORK IS DROPPING TRAFFIC

Scenario:

- Users cannot connect to a web application.
- Local tests work, but remote users fail.
- You need to determine if the server or network is dropping traffic.

Investigation Steps:

- Capture traffic on the server (tcpdump).
- Look for incoming SYN packets.
- Check if SYN-ACK is sent back.
- If no SYN reaches the server, the network is dropping traffic.
- If SYN reaches the server but no response, the server or firewall is the issue.
- Use packet analysis to provide evidence.

Key Takeaway:

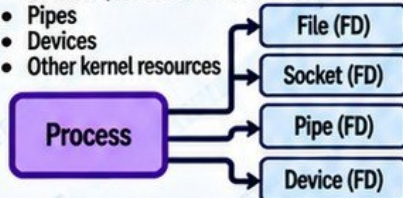


Packet capture gives you proof, not assumptions.

1 FILE DESCRIPTOR MODEL

- A file descriptor (FD) is a non-negative integer used by a process to access:

- Files
- Directories
- Sockets (network connections)
- Pipes
- Devices
- Other kernel resources



Everything is a file in Linux, including network connections.

2 SOFT VERSUS HARD LIMITS

- Each process has limits on the number of file descriptors it can open.
- Soft limit: the current limit for the process (can be changed).
- Hard limit: the maximum limit (ceiling).

Limit Type	Description	Can be changed?
Soft limit	Current limit	Yes (up to hard limit)
Hard limit	Maximum limit	Only by root

```

$ ulimit -n # show current soft limit
$ ulimit -Hn # show current hard limit
$ ulimit -n 65536 # set soft limit (temporary)
  
```

If a process reaches its limit, it cannot open new files or connections.

3 ULIMIT

- Use ulimit to view or set limits for the current shell session.
- Common options:
 - n : maximum number of open files
 - u : maximum number of processes
 - l : locked-in-memory size

```

$ ulimit -a # show all limits
$ ulimit -n # show open file limit
$ ulimit -n 65536 # set new soft limit
  
```

Changes made with ulimit are temporary and apply only to the current session.

4 /PROC/<PID>/LIMITS

- View the limits for a specific process.

```
$ cat /proc/<pid>/limits
```

Example output:

Limit	Soft	Hard
Max open files	1024	4096
Max processes	4096	4096
Max locked memory	65536	65536
...		

Useful for checking the actual limits of running services.

5 LSOF

- Use lsof to list open file descriptors for a process or system.

```

$ lsof -p <pid> # files opened by a process
$ lsof | wc -l # total open files
$ lsof -i # network sockets only
  
```

Useful options:

- p <pid> : show files for a process
- i : show network connections (sockets)
- n : no DNS resolution (faster)
- u <user> : files for a specific user

Helps identify descriptor leaks and which files/sockets are open.

6 SOCKET DESCRIPTORS

- Network connections use file descriptors (sockets).
- Each TCP/UDP connection consumes at least one file descriptor.
- High traffic services (web servers, databases) can exhaust descriptors quickly.

```

$ lsof -i -n | head
$ ss -tanp # show TCP connections
$ lsof -i :80 # connections on port 80
  
```

A large number of sockets can indicate high usage or a leak.

7 fs.file-max

- This is the system-wide maximum number of file descriptors.
- Check the current value:

```
$ cat /proc/sys/fs/file-max
```

- Change (temporary):

```
$ sudo sysctl -w fs.file-max=2097152
```

- Make permanent (in /etc/sysctl.conf):

```
fs.file-max = 2097152
```

This limit must be high enough for your workload.

8 DESCRIPTOR LEAKS

- A descriptor leak happens when a process opens files/sockets but does not close them.
- Over time, the process exhausts its limit.
- Common causes:
 - Application bugs
 - Unclosed network connections
 - Improper error handling
 - Third-party libraries

```

$ lsof -p <pid> | wc -l # count open files
$ watch -n 2 "lsof -p <pid> | wc -l" # monitor
  
```

Monitor file descriptor usage over time to detect leaks early.

9 SYSTEMD LIMITS

- Systemd services can have their own limits, independent of the user shell.
- Configure in a service file:

```

[Service]
LimitNOFILE=65536
LimitNPROC=4096
  
```

- Reload systemd and restart the service:

```

$ sudo systemctl daemon-reload
$ sudo systemctl restart <service>
  
```

Ensure systemd limits are high enough for high-traffic services.

10 PRODUCTION SCENARIO: "TOO MANY OPEN FILES"



Scenario:

- A web application starts returning errors: "Too many open files".
- New connections are failing.
- Existing connections may still work.
- Logs show EMFILE (too many open files).

Investigation Steps:

- 1 Check application and system limits (ulimit, /proc/<pid>/limits).
- 2 Use lsof to see what files/sockets are open.
- 3 Identify if there is a descriptor leak.
- 4 Increase limits (process and system-wide).
- 5 Fix the root cause in the application (close files/sockets).
- 6 Monitor to prevent recurrence.

Key Takeaway:



- Monitor file descriptor usage.
- Set appropriate limits.
- Fix leaks, don't just increase limits.

1 UID AND GID BEHAVIOR

- Every user has a User ID (UID) and Group ID (GID).
- UID 0 is root.
- A user can belong to multiple groups (supplementary groups).
- File ownership is based on UID and GID, not the username.



```
$ id
$ id <username>
$ groups <username>
```

Different users can have different permissions even if they belong to the same group.

2 PERMISSION EVALUATION

- Linux uses read (r), write (w), and execute (x) permissions.
- Permissions are evaluated in this order:
 - Owner
 - Group
 - Others
- Use `ls -l` to view permissions.

```
-rw-r--r--
Owner  Group  Others
rw-   r--   r--
```

```
$ ls -l <file>
$ stat <file> # detailed information
```

The most specific match is used (owner → group → others).

3 DIRECTORY EXECUTE PERMISSION

- Execute (x) on a directory does NOT allow running files.
- It allows you to access the directory contents (list files, enter, traverse).
- Without execute permission, you cannot access files inside the directory even if you have read permission.

```
$ ls -ld /path/to/dir
$ namei -l /path/to/file
```



You need execute permission on every directory in the path to reach a file.

4 ACLS (ACCESS CONTROL LISTS)

- ACLs provide more flexible permissions than traditional owner/group/others.
- Useful when you need to give specific users or groups access without changing ownership.

```
$ getfacl <file> # view ACLs
$ setfacl -m u:user:rw <file> # add ACL
$ setfacl -x u:user <file> # remove ACL
```

Example output:

```
# file: data.txt
# owner: appuser
# group: appgroup
user::rw-
user:deploy:rw-
group::r--
mask::rw-
other::r--
```



ACLs are evaluated in addition to traditional permissions.

5 LINUX CAPABILITIES

- Capabilities allow processes to perform specific privileged operations without full root access.
- Common capabilities:
 - CAP_NET_BIND_SERVICE (bind to ports < 1024)
 - CAP_SYS_ADMIN (administrative operations)
 - CAP_DAC_READ_SEARCH (bypass file permissions)

```
$ getcap <binary> # check capabilities
$ setcap cap_net_bind_service=ep <binary>
$ getpcaps <pid> # view process capabilities
```

Use capabilities instead of full root privileges to follow the principle of least privilege.

6 USEFUL COMMANDS

```
$ namei -l /path/to/file # check path permissions
$ getfacl /path/to/file # view ACLs
$ setfacl -m u:user:rw file # set ACL
$ getcap /path/to/binary # check capabilities
$ getpcaps <pid> # process capabilities
$ ls -l /path/to/file # check ownership/perm
$ stat /path/to/file # detailed file info
$ id <user> # check UID/GID
$ groups <user> # check group membership
```

These commands help you quickly identify permission, ownership and capability issues.

7 SERVICE-USER DIFFERENCES

- Services often run as dedicated system users (e.g. www-data, nginx, postgres).
- They may have different home directories, environments and group memberships.
- A command that works as root may fail for a service user due to missing permissions, paths or environment variables.

```
$ sudo -u <user> <command> # test as user
$ systemctl show <service> | grep User
```

Always test with the actual service user to reproduce permission issues.

8 SELINUX / APPARMOR AWARENESS

- SELinux and AppArmor can block access even when traditional permissions are correct.
- Check the security context or profile.

SELinux:

```
$ getenforce # check mode
$ ls -Z <file> # view context
$ ausearch -m avc -ts recent # check denials
```

AppArmor:

```
$ aa-status # check status
$ cat /var/log/syslog | grep apparmor
```

If access is denied, check SELinux/AppArmor logs for policy violations.

9 LEAST PRIVILEGE

- Grant only the minimum permissions required.
- Avoid running applications as root.
- Use dedicated service accounts.
- Use ACLs or capabilities instead of broad permissions.
- Regularly review and remove unnecessary access.



Benefits:

- Reduces security risk
- Limits the impact of compromise
- Prevents accidental damage
- Follows security best practices

Least privilege keeps your system secure and stable.

10 PRODUCTION SCENARIO: ROOT SUCCEEDS WHILE APPLICATION USER FAILS



Scenario:

- A web application cannot read a configuration file.
- The same command works when run as root.
- The service is failing with "Permission denied".
- The file exists and permissions look correct.

Investigation Steps:

- Check file ownership, permissions and ACLs.
- Verify directory execute permissions.
- Test with the actual service user.
- Check SELinux/AppArmor logs.
- Review Linux capabilities if needed.
- Apply least privilege fix and retest.

Key Takeaway:



- Root succeeding does not mean it is a permission issue.
- Always test with the application user and check all access controls (permissions, ACLs, capabilities, SELinux/AppArmor).

Kernel, sysctl and Host-Level Failures

Understand the host. Tune with care. Keep your systems stable.

Real Systems
Real Problems
Real Solutions

1 KERNEL MESSAGES

- The kernel logs important events, errors and warnings.
- Check kernel messages to identify hardware, driver, filesystem or memory issues.
- Kernel logs are usually viewed with dmesg or from /var/log/kern.log (or journalctl).



```
$ dmesg | tail -n 50 # recent kernel messages
$ dmesg -T           # human readable time
$ journalctl -k      # kernel logs via systemd
```

Kernel messages help you find the root cause of many host-level problems.

2 DMSG

- dmesg shows kernel ring buffer messages.
- Useful for boot issues, hardware detection, driver errors and kernel panics.
- Use filters to find specific errors.

```
$ dmesg | grep -i error
$ dmesg | grep -i fail
$ dmesg | less
```

Check dmesg when the system behaves unexpectedly, reboots or devices are not working.

3 /PROC FILESYSTEM

- /proc provides a virtual filesystem with kernel and process information.
- Useful for checking system limits, memory, CPU, processes and more.



```
$ ls /proc
$ cat /proc/meminfo # memory info
$ cat /proc/cpuinfo # CPU info
$ cat /proc/sys/kernel/hostname # hostname
```

/proc is a powerful source of real-time kernel and system information.

4 SYSCTL



- sysctl is used to view and modify kernel parameters at runtime.
- Parameters control behaviour like networking, memory, process limits and more.

```
$ sysctl -a # show all parameters
$ sysctl net.ipv4.ip_forward # view parameter
$ sysctl -w net.ipv4.ip_forward=1 # set temporarily
$ cat /etc/sysctl.conf # persistent configuration
```

Use sysctl to tune the kernel, but always understand the impact.

5 KERNEL LIMITS



- The kernel enforces system-wide limits on resources like processes, files, memory and sockets.
- Check important limits:

```
$ cat /proc/sys/fs/file-max # max file handles
$ cat /proc/sys/kernel/pid_max # max PIDs
$ cat /proc/sys/kernel/threads-max # max threads
$ cat /proc/sys/vm/max_map_count # memory maps
```

If these limits are too low, applications may fail under load.

6 NETWORK TUNABLES



- Network performance and behaviour can be tuned using sysctl parameters.
- Common network tunables:

```
$ sysctl net.ipv4.ip_forward
$ sysctl net.core.somaxconn
$ sysctl net.ipv4.tcp_max_syn_backlog
$ sysctl net.ipv4.tcp_fin_timeout
$ sysctl net.ipv4.tcp_tw_reuse
```

Incorrect network tuning can cause connectivity issues, packet loss or poor performance.

7 PID LIMITS



- The kernel limits the maximum number of processes (PIDs).
- If the PID limit is reached, new processes cannot be created.

```
$ cat /proc/sys/kernel/pid_max # max PID value
$ ps -e | wc -l # current process count
$ cat /proc/sys/kernel/threads-max # max threads
```

When the system is under heavy load, PID exhaustion can cause application and service failures.

8 MEMORY TUNABLES



- Kernel memory settings control how the system uses RAM, swap and caches.
- Important memory tunables:

```
$ sysctl vm.swappiness
$ sysctl vm.overcommit_memory
$ sysctl vm.dirty_ratio
$ sysctl vm.dirty_background_ratio
$ sysctl vm.max_map_count
```

Improper memory tuning can lead to out-of-memory issues or poor performance.

9 KERNEL PANIC AND SOFT LOCKUP

- A kernel panic occurs when the kernel cannot recover from a critical error.
- A soft lockup happens when a CPU is stuck and cannot schedule other tasks.
- Check logs for panic or lockup messages.

```
$ dmesg | grep -i panic
$ dmesg | grep -i lockup
$ journalctl -k | grep -i lockup
```



Kernel panics and lockups usually indicate hardware, driver or severe resource issues.

10 DANGEROUS TUNING PRACTICES

- Do not copy random tuning values from the internet.
- Do not set values without understanding the impact.
- Avoid extreme values that can make the system unstable.
- Test changes in a non-production environment first.
- Monitor the system after changes.



Wrong tuning can cause instability, performance issues or even system crashes.

11 PRODUCTION SCENARIO

Application failure caused by host-level limits



Scenario:

- A web application starts failing under high load.
- Logs show: "cannot create new thread" or "cannot fork".
- The system has reached the PID or thread limit.
- Increasing kernel limits and adjusting sysctl values resolves the issue.

Investigation Steps:

- 1 Check application and system logs.
- 2 Verify current kernel limits (/proc and sysctl).
- 3 Identify which resource is exhausted (PIDs, threads, memory, file handles).
- 4 Increase limits appropriately.
- 5 Monitor to confirm the issue is resolved.
- 6 Document the changes.

Key Takeaway:

- Host-level limits can directly impact application availability.
- Monitor key kernel parameters.
- Tune carefully and test before applying in production.

Different views. Same system. Find the real problem.

1 HOST VERSUS CONTAINER PERSPECTIVE

- The host and container can show different views of the same system.
- The host may look healthy while the container is resource-starved.
- Always check from both perspectives.

Host View



- Full system resources
- All processes
- Overall system health

Container View



- Limited resources
- Isolated processes
- May be experiencing issues

Always check both host and container views during troubleshooting.

2 PID, NETWORK AND MOUNT NAMESPACES

- Namespaces isolate resources for containers.
- Each container has its own view of:
 - PID namespace (processes)
 - Network namespace (network interfaces)
 - Mount namespace (file system)
 - Other namespaces: IPC, UTS, User

```
# Check namespaces for a process
$ ls -l /proc/<pid>/ns
# Example output
pid -> pid:[4026533123]
net -> net:[4026531992]
mnt -> mnt:[4026531840]
uts -> uts:[4026533122]
ipc -> ipc:[4026533121]
```

Namespaces provide isolation. Understanding them helps you debug container issues.

3 CGROUPS (CONTROL GROUPS)

- cgroups limit, account and isolate resource usage for containers.
- Control resources like CPU, memory, I/O and more.
- Modern systems use cgroup v2 (unified hierarchy).

```
/sys/fs/cgroup/
├── cpu/
├── memory/
├── io/
├── pids/
└── ...
```



cgroups ensure one container cannot consume all host resources.

4 CPU THROTTLING

- Containers can be limited to a specific amount of CPU.
- When the limit is reached, processes are throttled by the kernel.
- This can cause performance issues even though the host has free CPU.

```
# Check CPU cgroup (v2)
$ cat /sys/fs/cgroup/cpu.max
# Check CPU usage and throttling
$ cat /sys/fs/cgroup/cpu.stat
```

If a container is slow, check for CPU throttling before assuming an application issue.

5 MEMORY LIMITS AND OOM BEHAVIOR

- Containers have memory limits enforced by cgroups.
- When memory is exhausted, the kernel may kill processes (OOM).
- This can happen even if the host has plenty of free memory.

```
# Check memory limit (v2)
$ cat /sys/fs/cgroup/memory.max
# Check current usage
$ cat /sys/fs/cgroup/memory.current
# Check memory events
$ cat /sys/fs/cgroup/memory.events
```

A container can be killed by the OOM killer even when the host is healthy.

6 NAMESPACE INSPECTION AND NSENTER

- You can inspect a container's namespaces and even enter them using nsenter.
- This helps you debug from inside the container's view.

```
# List namespaces for a container process
$ ls -l /proc/<pid>/ns
# Enter container namespaces
$ sudo nsenter -t <pid> -m -u -i -n -p
# Or enter only network namespace
$ sudo nsenter -t <pid> -n
# Run a shell inside the container's namespaces
$ sudo nsenter -t <pid> -m -u -i -n -p /bin/bash
```

nsenter is powerful for troubleshooting without using docker exec or kubectl exec.

7 /PROC AND CONTAINER PROCESS VISIBILITY

- /proc shows information about running processes, but inside a container you only see processes in that namespace.
- Use /proc to check CPU, memory, limits and process details.

```
# List processes inside container namespace
$ ps aux
# Check cgroup info for a process
$ cat /proc/<pid>/cgroup
# View mount points
$ cat /proc/<pid>/mountinfo
```

Process visibility is limited to the container's namespace. Use the host or nsenter to see more.

8 USEFUL COMMANDS

Purpose	Command
Check container cgroups	cat /proc/<pid>/cgroup
Check namespaces	ls -l /proc/<pid>/ns
Check CPU limits (v2)	cat /sys/fs/cgroup/cpu.max
Check memory limits (v2)	cat /sys/fs/cgroup/memory.max
Check memory usage	cat /sys/fs/cgroup/memory.current
Check cgroup stats	cat /sys/fs/cgroup/*/*stat
Enter namespaces	nsenter -t <pid> -m -u -i -n -p
View processes (host)	ps aux grep <container>
View processes (container)	docker top <container>
Inspect container	docker inspect <container>

These commands help you investigate resource and isolation issues.

9 PRODUCTION SCENARIO

Host looks healthy while container is resource-starved

Scenario:

- A web application in a container is slow or unresponsive.
- The host has plenty of CPU and memory.
- Inside the container, processes are being CPU throttled or killed due to memory limits.
- The application logs show timeouts or unexpected exits.
- Investigation reveals cgroup limits are too low.

Solution:

- Check container resource limits.
- Monitor cgroup metrics.
- Adjust limits or optimize the application.
- Validate after changes.

Always investigate from both host and container perspectives to find the real root cause.



1 SYSTEM-CALL TRACING

- System calls are the interface between user space and the kernel.
- Tracing system calls helps you understand what an application is doing.
- Useful for debugging performance issues, errors and unexpected behaviour.

```
# trace a command
strace -o strace.log <command>
# trace a running process
strace -p <pid>
# show only specific syscalls
strace -e trace=read,write,open -p <pid>
```



strace shows you the actual system calls and their return values.

2 STRACE IN PRACTICE

- Common use cases:
 - Find which files are being accessed.
 - Understand where an application is spending time (e.g. I/O, network).
 - Identify failing system calls.
 - Investigate slow startups.

```
strace -tt -T -p <pid> # show time and duration
strace -f -o app.log <command> # follow child processes
strace -e trace=file <command> # trace file-related calls
strace -c <command> # summary of syscalls
```

Look for repeated or slow system calls, errors (ENOENT, EACCES) and time spent in specific calls.

3 CPU PROFILING WITH PERF

- perf is a powerful tool for profiling CPU usage, scheduling and more.
- Helps identify where the CPU is being spent in your application or the kernel.
- Works with sampling to minimize overhead.

```
# record CPU profile
perf record -F 99 -g <command>
# analyze the profile
perf report
# quick top-like view
perf top
```



perf shows you which functions and code paths are consuming CPU time.

4 FLAME GRAPH CONCEPTS

- Flame graphs give a visual representation of where CPU time is being spent.
- Each box represents a function.
- Wider boxes mean more CPU time.
- Helps you quickly find performance bottlenecks.



Flame Graph

Flame graphs make it easy to see hot code paths at a glance.

5 eBPF OBSERVABILITY

- eBPF allows low-overhead, in-kernel observability without modifying application code.
- Can trace syscalls, network, disk I/O, scheduling and more.
- Tools: bcc, bpftrace, pixie, tracee, Cilium.

```
# trace syscalls with bpftrace
sudo bpftrace -e 'tracepoint:syscalls:sys_enter_* { @[probe] = count(); }'
# trace slow syscalls (> 10ms)
sudo bpftrace -e 'tracepoint:syscalls:sys_exit_* /args->ret > 10000000/ { printf("%s %d\n", probe, args->ret); }'
```

eBPF gives deep insights with low overhead and is safe for production when used carefully.

6 OFF-CPU VERSUS ON-CPU ANALYSIS

- On-CPU: time when the process is actually using the CPU.
- Off-CPU: time spent waiting (I/O, locks, network, scheduling, etc).
- High off-CPU time usually indicates blocking operations or resource contention.

On-CPU

- Running
- Using CPU cycles
- Shows active work

VS

Off-CPU

- Waiting
- Blocked (I/O, locks)
- Not using CPU
- Often the real bottleneck

Both on-CPU and off-CPU analysis are important to find the real performance issue.

7 SYSCALL LATENCY

- High syscall latency can indicate problems with disk, network, filesystems or contention.
- Use tools to measure the time spent in system calls.

```
# time each syscall with strace
strace -tt -T -p <pid>
# trace slow syscalls with bpftrace
sudo bpftrace -e 'tracepoint:syscalls:sys_exit_* /nsecs > 10000000/ { printf("%s %d ms\n", probe, nsecs/1000000); }'
```

Look for syscalls that consistently take longer than expected.

8 BLOCKED PROCESSES

- Processes can be blocked on I/O, locks, network or other resources.
- Use different tools to identify blocked processes.

```
# show process states
ps -eo pid,stat,comm
# find blocked processes (D state)
ps -eo pid,stat,comm | grep '^D'
# use perf to see where they are blocked
perf sched timehist
# use bpftrace to trace off-CPU time
sudo bpftrace -e 'offcputime { @[comm] = sum(arg0); }'
```

Blocked processes can significantly impact application performance.

9 PRODUCTION SAFETY AND TRACING OVERHEAD

- Tracing tools can add overhead, especially on busy systems.
- Use sampling instead of tracing every event.
- Limit the scope (specific process, PID, time window).
- Be careful with eBPF programs and kernel versions.
- Always test in a non-production environment first when possible.

Best Practices:

- ✓ Use sampling (e.g. perf -F 99)
- ✓ Trace only what you need
- ✓ Limit time and scope
- ✓ Monitor system impact
- ✓ Have a rollback plan



Get the insights you need without causing new problems.

10 PRODUCTION SCENARIO: FINDING WHERE APPLICATION TIME IS ACTUALLY BEING SPENT



Scenario:

- A web application is slow under load.
- CPU usage is moderate.
- No obvious errors in logs.
- The team needs to find where the time is being spent (CPU, I/O, network or locks).

Investigation Steps:

- 1 Use strace to check system calls and identify slow operations.
- 2 Use perf to profile CPU usage and generate a flame graph.
- 3 Use eBPF tools to analyze on-CPU and off-CPU time.
- 4 Check for blocked processes and resource contention.
- 5 Correlate findings and identify the real bottleneck.
- 6 Implement a fix and verify performance improvement.

Key Takeaway:

- The application may look fine, but the real issue is hidden.
- Use multiple tools and perspectives.
- Measure, don't guess.
- Find the real cause, not just the symptoms.

Capstone: Multi-Symptom Linux Production Incident

Real Systems
Real Problems
Real Solutions

Think like a senior engineer. Investigate. Correlate. Find the real cause. Prevent it.

1 INCIDENT OVERVIEW

- Multiple symptoms appear at the same time:
 - API latency rises
 - Load average spikes
 - Some requests time out
 - Disk latency increases
 - Memory pressure appears
 - TCP connections accumulate
 - Service begins restarting



Your task is to investigate, find the root cause and restore stability with the least risk.

2 DEFINE THE BLAST RADIUS

- Identify what is impacted:
 - Which services?
 - Which users?
 - Which environments (prod, staging)?
 - Is the entire system affected or only specific components?
 - Communicate the impact clearly.



```
# check affected services
$ systemctl status <service>
$ curl -w "%{time_total}\n" <endpoint>
$ ss -s
$ journalctl -u <service> --since "1 hour ago"
```

Understand the scope before making changes.

3 BUILD AN INCIDENT TIMELINE

- Collect timestamps from all available sources:
 - System logs (journalctl, /var/log)
 - Application logs
 - Metrics (CPU, memory, disk, network)
 - Service restarts
 - Changes (deployments, config, etc.)



```
# view logs with timestamps
$ journalctl --since "2 hours ago"
$ journalctl -u <service> --no-pager
$ grep -r "ERROR" /var/log/
$ dmesg -T | tail -n 50
```

A clear timeline helps you see cause and effect.

4 CORRELATE SYSTEM EVIDENCE

- Look at all key areas and find patterns:
 - CPU (high usage?)
 - Memory (pressure, swaps?)
 - Disk (high latency, I/O wait?)
 - Network (errors, high connections?)
 - Service behaviour (restarts, failures?)



```
# useful commands
$ top / htop
$ free -h
$ iostat -xz 1
$ free -l n
$ ss -s | ss -tanp
$ dmesg -T | grep -i error
```

Look for correlations, not just isolated metrics.

5 SEPARATE SYMPTOMS FROM CAUSAL SIGNALS

- Not every problem is the root cause.
- Identify which signals are:
 - Symptoms (effects)
 - Causal signals (likely causes)
 - Ignore noise and focus on evidence that explains multiple symptoms.

Symptoms

- High CPU
- API latency
- Service restarts
- Timeouts

Causal Signals

- Disk I/O saturation
- Memory exhaustion
- Connection leak
- Bug or configuration change

The real cause usually explains multiple symptoms.

6 IDENTIFY THE PRIMARY ROOT CAUSE

- Use evidence to find the most likely root cause.
- Validate your theory.
- Rule out other possibilities.
- Check recent changes (deployments, kernel updates, etc.).



```
# examples of investigation
$ git log --oneline | head
$ cat /etc/*release
$ systemctl list-units --failed
$ grep -i "oom" /var/log/messages
$ lsof | grep deleted
```

Be confident in the root cause before taking action.

7 CHOOSE THE SAFEST MITIGATION

- Consider the impact of each option.
- Prefer low-risk, reversible changes.
- Examples:
 - Restart a specific service
 - Increase a limit temporarily
 - Scale resources
 - Roll back a recent change



Stabilize the system without making the problem worse.

8 VERIFY RECOVERY

- Confirm that all symptoms are resolved.
- Check metrics and logs.
- Ensure the service is stable.
- Monitor for a period of time.

```
# verify after mitigation
$ curl -w "%{time_total}\n" <endpoint>
$ systemctl status <service>
$ top
$ ss -s
$ journalctl -u <service> --since "10 minutes ago"
```

Recovery is not complete until the system is stable over time.

9 DETERMINE MONITORING GAPS

- What should have alerted you earlier?
- Which metrics or logs were missing?
- Do you need better dashboards, alerts or log collection?



Examples:

- Disk latency alerts
- Memory usage and swap alerts
- TCP connection count alerts
- Service restart alerts
- Application error rate alerts

Turn this incident into better observability.

10 WRITE PREVENTION ACTIONS



- Fix the underlying issue.
- Improve monitoring and alerting.
- Update runbooks.
- Add automated tests or checks.
- Review capacity and limits.
- Share learnings with the team.

Prevent the same incident from happening again.

11 SENIOR-LEVEL INCIDENT CONCLUSION



- Write a clear incident summary:
 - What happened?
 - What was the root cause?
 - How was it fixed?
 - What could have been detected earlier?
 - What will be done differently?
 - Key lessons for the team.



“A great engineer not only fixes problems, but also prevents them.”
— VERIQTA

Close the incident with clarity, accountability and long-term improvement.